2026

MODUL PRAKTIKUM I

INTRODUCTION TO PYTORCH

OPTIMIZED VERSION

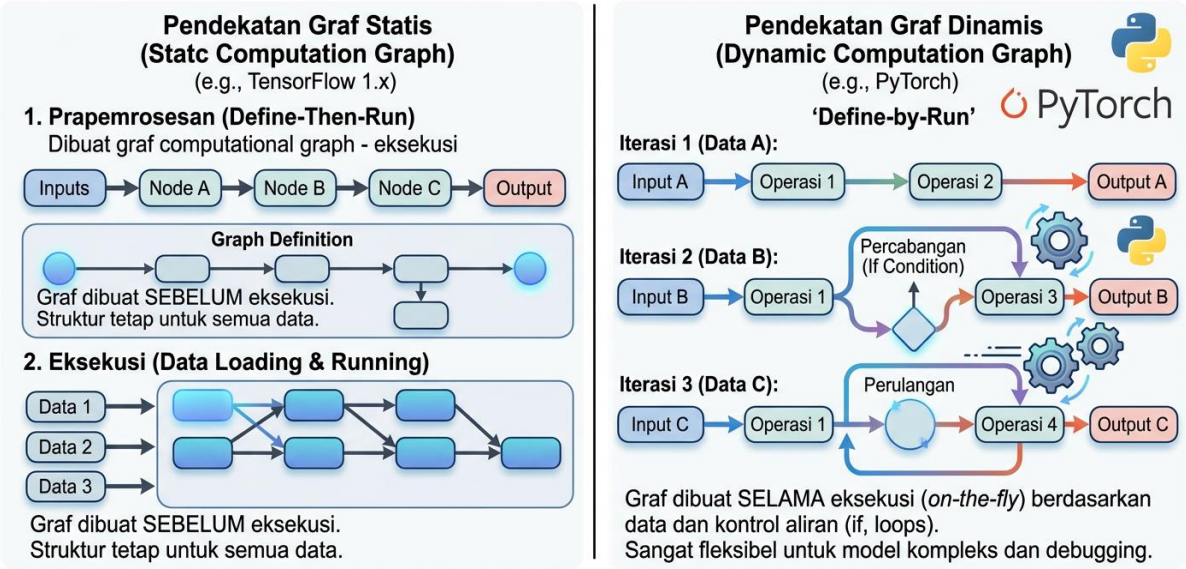
Disusun Oleh:
Ryan F F Hakim

Dipersembahkan Untuk:
Sains Data

Mata Kuliah
Deep Learning

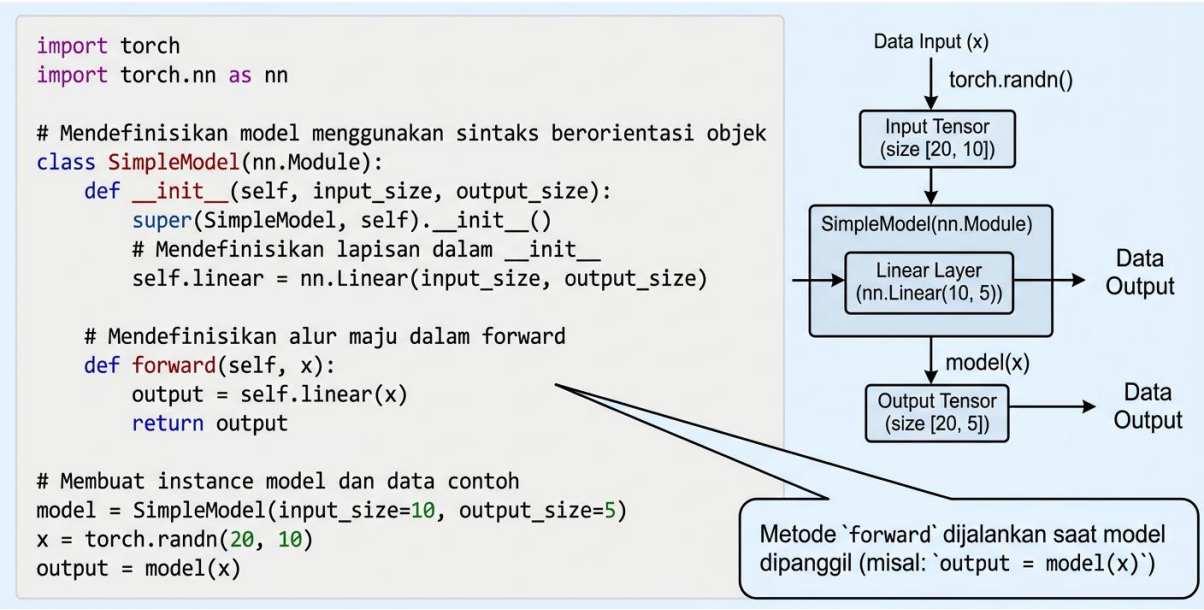
PENDAHULUAN

PyTorch adalah salah satu *framework machine learning* dan *deep learning open-source* yang paling populer saat ini, dikembangkan secara utama oleh tim riset kecerdasan buatan dari Meta (sebelumnya Facebook AI Research). Sejak diluncurkan, PyTorch telah menjadi standar utama baik di dunia akademis maupun industri karena menawarkan fleksibilitas yang luar biasa serta kemudahan dalam mengekspresikan model-model *deep learning* yang kompleks. Berbeda dengan beberapa kerangka kerja lawas yang mengharuskan pengguna mendefinisikan graf komputasi statis di awal, PyTorch mengusung konsep *dynamic computation graph* atau *eager execution*. Pendekatan ini memungkinkan para peneliti dan pengembang untuk mengeksekusi perhitungan secara langsung saat kode dijalankan, sehingga arsitektur jaringan dapat dimodifikasi secara dinamis di tengah proses iterasi.



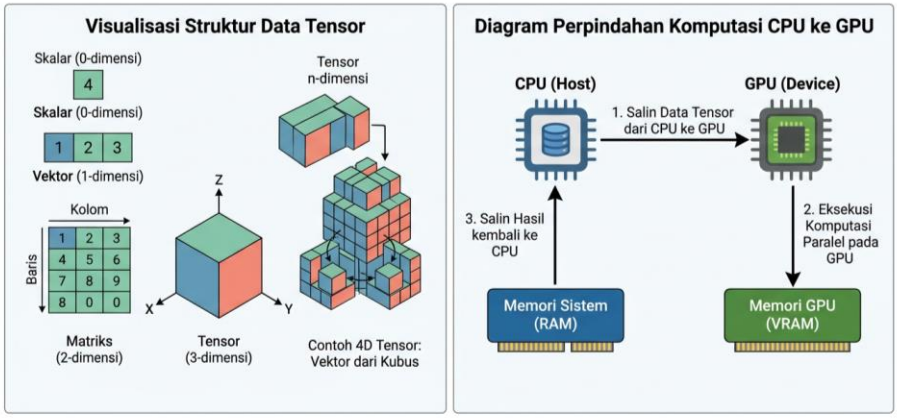
Gambar 1.1: Ilustrasi Dynamic Computation Graph pada PyTorch yang memungkinkan eksekusi perintah secara fleksibel dibandingkan pendekatan graf statis.

Salah satu keunggulan utama PyTorch yang membuatnya sangat cocok digunakan dalam modul praktikum adalah desainnya yang sangat *Pythonic*. Penulisan kode di dalam PyTorch dirancang agar terasa alami dan intuitif bagi siapa saja yang sudah terbiasa dengan bahasa pemrograman Python. Fleksibilitas ini memungkinkan PyTorch terintegrasi secara mulus dengan ekosistem ilmu data Python lainnya, seperti NumPy, SciPy, dan Pandas. Selain itu, karena pendekatannya yang dinamis, proses *debugging* atau pencarian kutu menjadi jauh lebih mudah. Mahasiswa atau praktikan dapat menggunakan alat *debugging* standar bawaan Python (seperti fungsi print() sederhana) di dalam lapisan model jaringan saraf untuk memeriksa nilai *loss*, akurasi, atau dimensi data pada tahap komputasi mana pun.



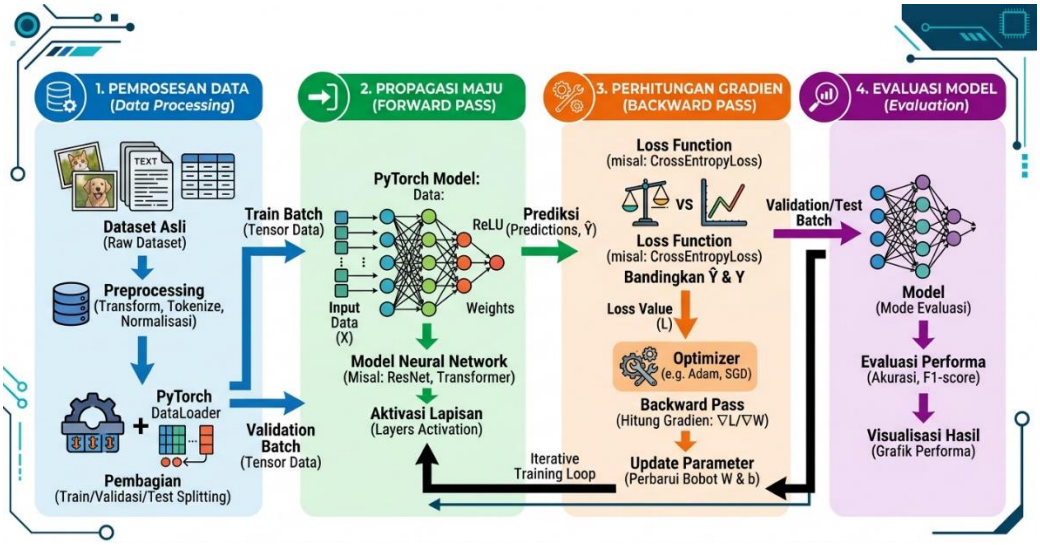
Gambar 1.2: Contoh struktur kode PyTorch yang sederhana dan Pythonic, menunjukkan betapa mudahnya mendefinisikan model menggunakan sintaks berorientasi objek.

Di jantung arsitektur PyTorch terdapat dua komponen paling fundamental: struktur data **Tensor** dan mesin diferensiasi otomatis yang dikenal sebagai **Autograd**. Tensor dalam PyTorch memiliki struktur dan cara kerja yang sangat mirip dengan *array* n-dimensi pada NumPy, namun memiliki satu keunggulan krusial: Tensor dirancang agar dapat dioperasikan di atas *Graphics Processing Unit* (GPU) untuk mengakselerasi proses komputasi matematis yang berat. Sementara itu, fitur *Autograd* bekerja di belakang layar untuk mencatat seluruh operasi matematika yang dilakukan pada Tensor, lalu menghitung gradien secara otomatis. Fitur ini memfasilitasi algoritma *backpropagation* secara instan, membebaskan pengguna dari keharusan menurunkan rumus kalkulus kompleks secara manual ketika memperbarui bobot model.



Gambar 1.3: Representasi visual struktur data Tensor dari 1-dimensi (vektor) hingga n-dimensi, serta diagram perpindahan komputasi dari CPU ke GPU.

Dalam implementasi arsitekturnya, PyTorch menyediakan paket tingkat tinggi yaitu torch.nn yang berisi kumpulan modul siap pakai untuk membangun *neural network*. Paket ini menyediakan berbagai macam lapisan jaringan (*layers*), fungsi aktivasi, hingga fungsi kerugian (*loss functions*) yang mengabstraksi kerumitan kode dasar. Melalui modul praktikum *deep learning* ini, Anda akan diajak untuk memahami alur kerja esensial PyTorch selangkah demi selangkah. Dimulai dari manipulasi dasar Tensor, menyusun arsitektur jaringan saraf, menyiapkan saluran pemuatan data (*data pipeline*), hingga melakukan siklus pelatihan dan evaluasi model. Penguasaan terhadap PyTorch akan memberikan fondasi yang sangat kuat bagi Anda untuk mengeksplorasi berbagai inovasi kecerdasan buatan, baik untuk keperluan riset akademis maupun implementasi profesional di industri teknologi.



Gambar 1.4: Alur kerja umum (pipeline) pelatihan Deep Learning menggunakan PyTorch, mulai dari pemrosesan data, propagasi maju (forward pass), perhitungan gradien (backward pass), hingga evaluasi.

DEPENDENSI

CODE

```
!pip install torch
```

CODE

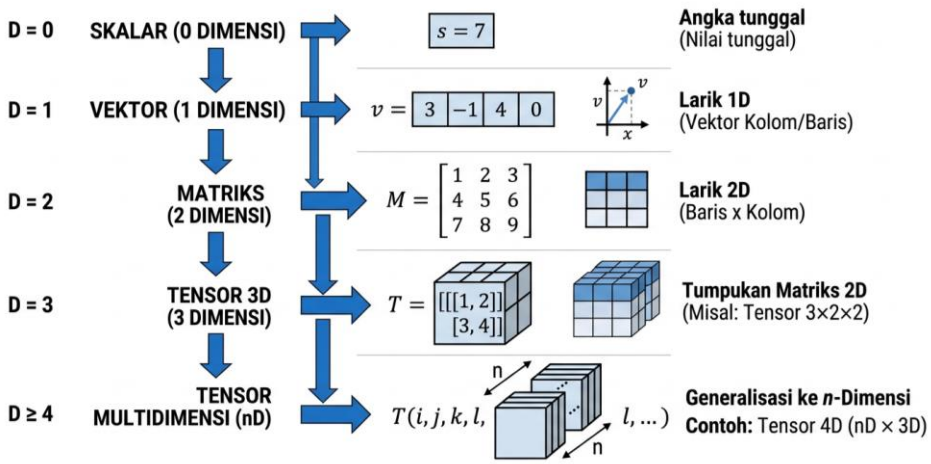
```
import torch
```

CODE

```
# Verifikasi instalasi
print(f"PyTorch Version : {torch.__version__}")
print(f"CUDA Available  : {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"CUDA Device     : {torch.cuda.get_device_name(0)}")
    print(f"CUDA Version    : {torch.version.cuda}")
```

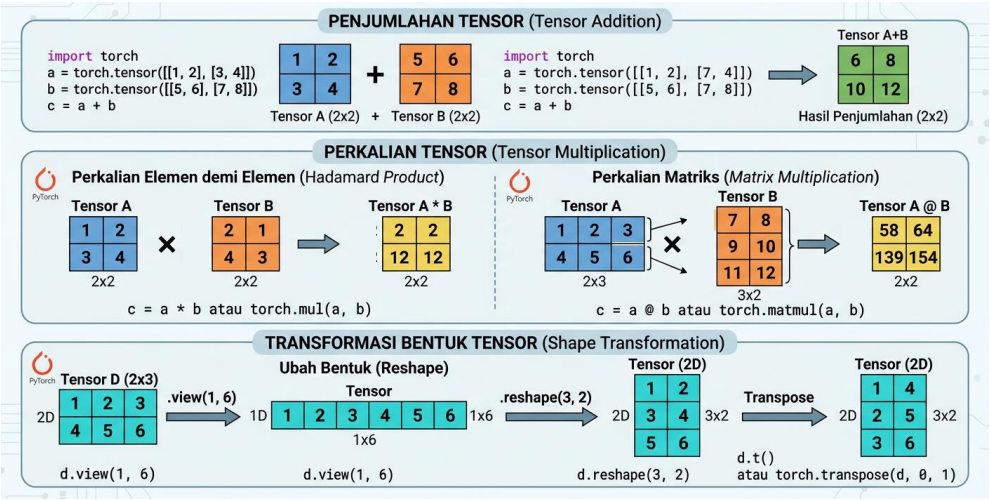
DASAR-DASAR TENSOR DAN OPERASI PYTORCH

Secara umum, tensor dapat didefinisikan sebagai struktur data multidimensi yang digunakan untuk menyimpan nilai-nilai numerik dalam susunan teratur. Tensor dapat dipandang sebagai perluasan dari skalar, vektor, dan matriks ke dimensi yang lebih tinggi. Tensor berdimensi nol disebut skalar, tensor berdimensi satu disebut vektor, dan tensor berdimensi dua disebut matriks. Sementara itu, tensor berdimensi tiga atau lebih digunakan untuk merepresentasikan data yang lebih kompleks, seperti citra berwarna, urutan data temporal, maupun sekumpulan sampel yang diproses secara bersamaan. Dengan pengertian tersebut, tensor bukan hanya sekadar wadah data, melainkan juga sarana utama untuk merepresentasikan informasi dalam bentuk yang dapat diolah secara matematis.



Gambar 2.2: Hierarki representasi data numerik dari skalar, vektor, matriks, hingga tensor multidimensi.

PyTorch menggunakan tensor sebagai inti dari hampir seluruh operasinya karena tensor memungkinkan pemrosesan numerik dilakukan secara efisien dan terstruktur. Berbagai operasi dasar, seperti penjumlahan, pengurangan, perkalian, pembagian, pengubahan bentuk data, pengambilan elemen tertentu, hingga operasi aljabar linear, dijalankan melalui tensor. Kemampuan ini menjadikan tensor sangat sesuai untuk kebutuhan Deep Learning, karena model jaringan saraf pada dasarnya tersusun atas serangkaian transformasi matematis terhadap data numerik. Dengan memahami tensor, hubungan antara representasi data dan proses komputasi dalam model dapat dipahami secara lebih jelas.



Gambar 2.3: Contoh operasi dasar tensor pada PyTorch, seperti penjumlahan, perkalian, dan transformasi bentuk data.

CODE

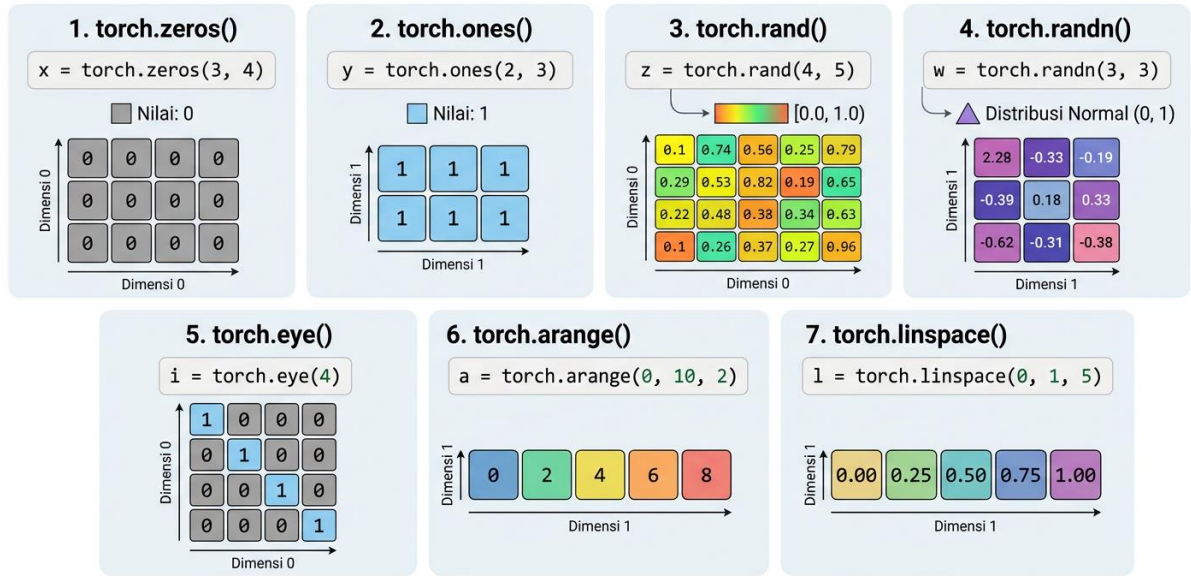
```
# Dari list Python
tensor_a = torch.tensor([1, 2, 3, 4, 5])
print(f"Tensor dari list  : {tensor_a}")
print(f"Shape             : {tensor_a.shape}")
print(f"Dtype             : {tensor_a.dtype}")
```

CODE

```
# Dari NumPy array
np_array = np.array([[1.0, 2.0], [3.0, 4.0]])
tensor_b = torch.from_numpy(np_array)
print(f"\nTensor dari NumPy:\n{tensor_b}")
```

CODE

```
# Tensor khusus
zeros = torch.zeros(3, 4)          # Tensor nol
ones = torch.ones(2, 3)            # Tensor satu
rand = torch.rand(3, 3)            # Uniform random [0, 1)
randn = torch.randn(3, 3)          # Normal random N(0, 1)
eye = torch.eye(4)                 # Matriks identitas
arange = torch.arange(0, 10, 2)    # Tensor dengan step
linspace = torch.linspace(0, 1, 5) # Tensor dengan spasi linear
```



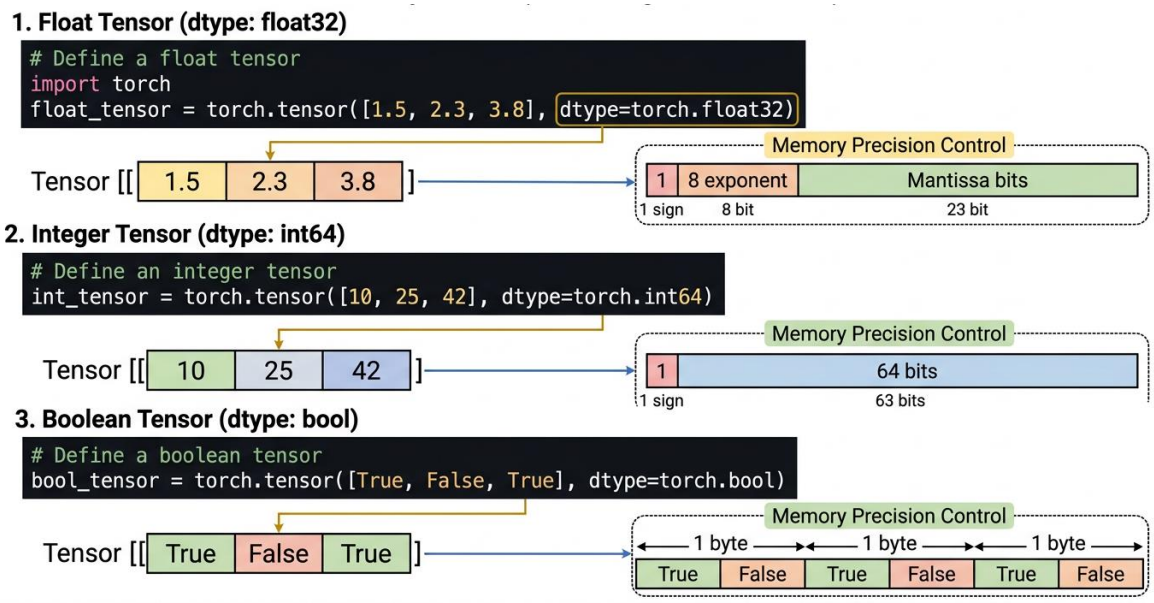
Gambar 2.4: Visualisasi pembentukan struktur data tensor melalui `torch.zeros()`, `torch.ones()`, `torch.rand()`, `torch.randn()`, `torch.eye()`, `torch.arange()`, `torch.linspace()`

CODE

```
print(f"\nZeros (3x4):\n{zeros}")
print(f"\nRandom Normal (3x3):\n{randn}")
print(f"\nIdentity (4x4):\n{eye}")
print(f"\nArange    : {arange}")
print(f"Linspace  : {linspace}")
```

CODE

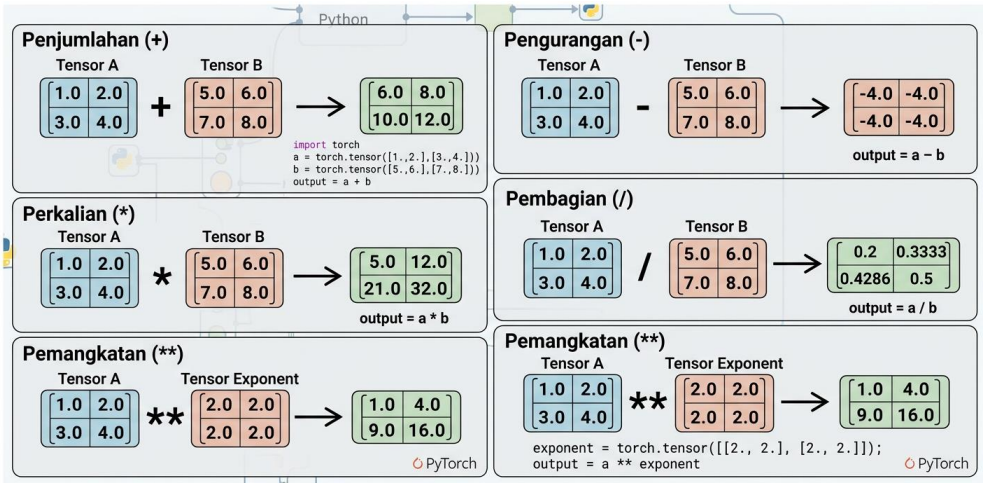
```
# Tensor dengan dtype spesifik
tensor_float = torch.tensor([1, 2, 3], dtype=torch.float32)
tensor_int = torch.tensor([1.5, 2.7], dtype=torch.int64)
tensor_bool = torch.tensor([1, 0, 1], dtype=torch.bool)

print(f"\nFloat tensor : {tensor_float} | dtype: {tensor_float.dtype}")
print(f"Int tensor   : {tensor_int} | dtype: {tensor_int.dtype}")
print(f"Bool tensor  : {tensor_bool} | dtype: {tensor_bool.dtype}")
```



Gambar 2.5: Pendefinisian tensor dengan spesifikasi tipe data (dtype) eksplisit untuk mengontrol presisi memori dan jenis nilai (Float, Integer, dan Boolean).

CODE

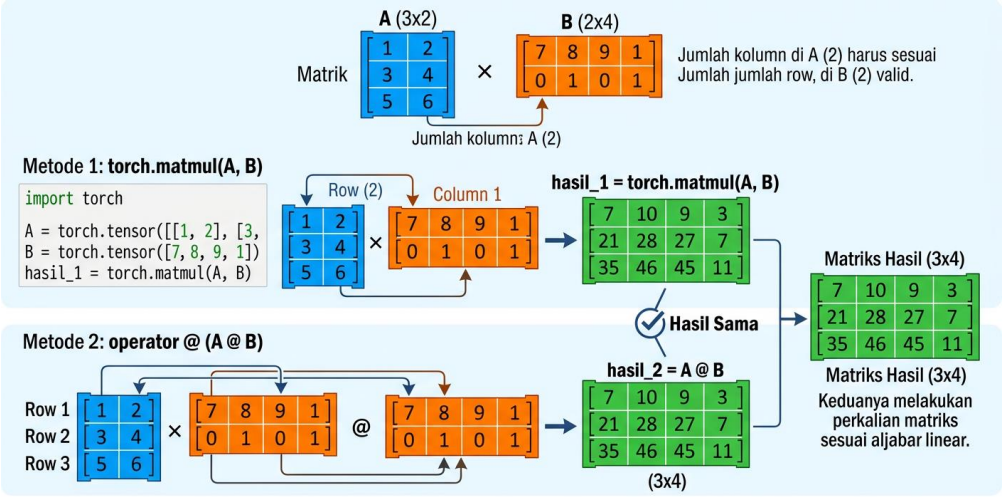
```
# 2. Operasi aritmatika

a = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
b = torch.tensor([[5.0, 6.0], [7.0, 8.0]])

# Operasi element-wise
print("Penjumlahan    : \n", a + b)
print("Pengurangan    : \n", a - b)
print("Perkalian e-w  : \n", a * b)
print("Pembagian      : \n", a / b)
print("Pangkat        : \n", a ** 2)
```



Gambar 2.6: Contoh kalkulasi matriks menggunakan operator standar PyTorch untuk melakukan penjumlahan, pengurangan, perkalian, pembagian, dan pemangkatan pada setiap elemen tensor.

CODE

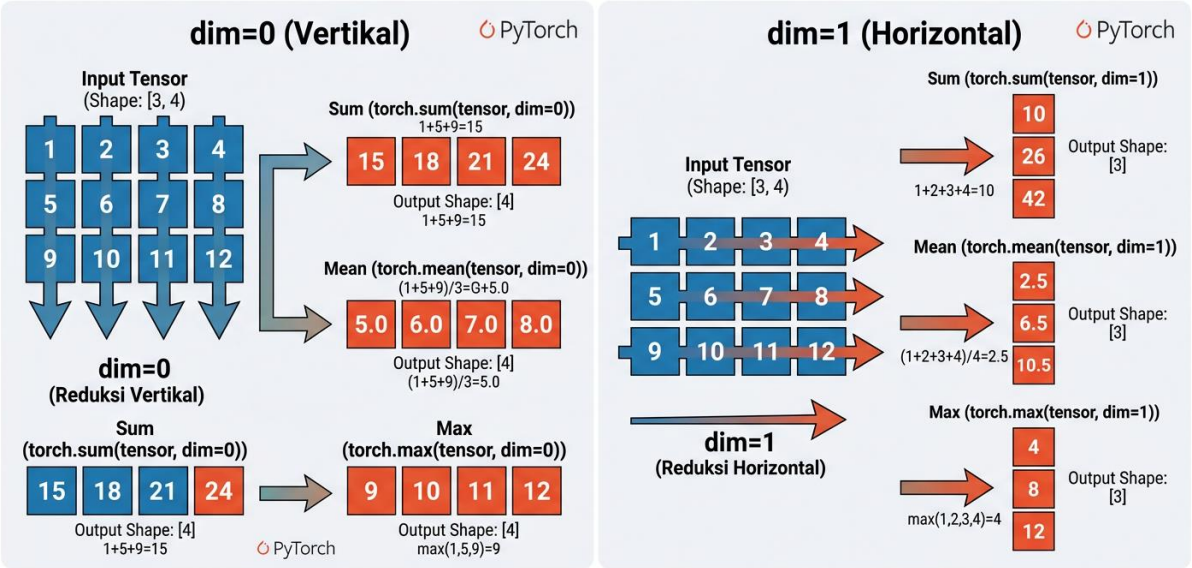
```
# Perkalian matriks (dot product)
mat_mul = torch.matmul(a, b)
mat_mul2 = a @ b                    # bisa juga seperti ini menggunakan "@"
print(f"\nPerkalian Matriks (a @ b):\n{mat_mul}")
print()
print(mat_mul2)
```



Gambar 2.7: Implementasi perkalian matriks (matrix multiplication) menggunakan fungsi `torch.matmul` dan operator "@" yang mengikuti aturan aljabar linear.

CODE

```
# Operasi agregasi
x = torch.tensor([[1.0, 2.0, 3.0],
                  [4.0, 5.0, 6.0]])

print(f"\nSum          : {x.sum()}")
print(f"Sum axis=0   : {x.sum(dim=0)}")       # Sepanjang baris
print(f"Sum axis=1   : {x.sum(dim=1)}")       # Sepanjang kolom
print(f"Mean         : {x.mean()}")
print(f"Max          : {x.max()}")
print(f"Argmax       : {x.argmax()}")
print(f"Min axis=1   : {x.min(dim=1)}")
```



Gambar 2.8: Reduksi dimensi tensor melalui operasi sum, mean, dan max pada PyTorch, mengilustrasikan cara kerja parameter dim=0 (vertikal) dan dim=1 (horizontal).

CODE

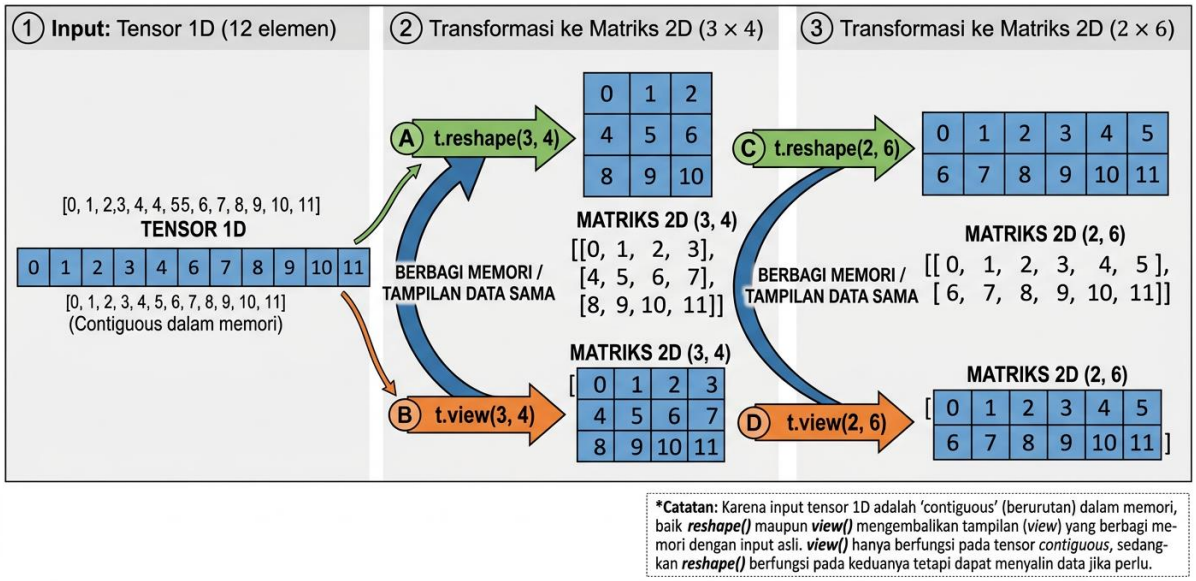
```
# 3. Reshaping dan Indexing

t = torch.arange(12).float()
print(f"\nOriginal     : {t}")
print(f"Shape        : {t.shape}")
```

CODE

```
# Reshape
reshaped = t.reshape(3, 4)
print(f"\nReshaped (3x4):\n{reshaped}")
```

CODE

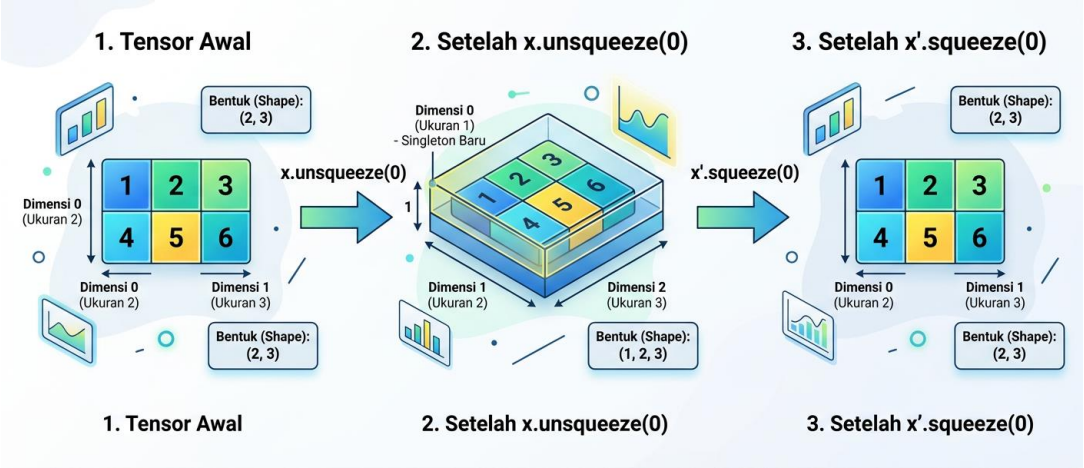
```
# View (shared memory)
viewed = t.view(2, 6)
print(f"\nViewed (2x6):\n{viewed}")
```



Gambar 2.9: Ilustrasi proses transformasi struktur data menggunakan fungsi `reshape()` dan `view()` pada tensor 1D berukuran 12 elemen menjadi bentuk matriks 2D (3 × 4 dan 2 × 6).

CODE

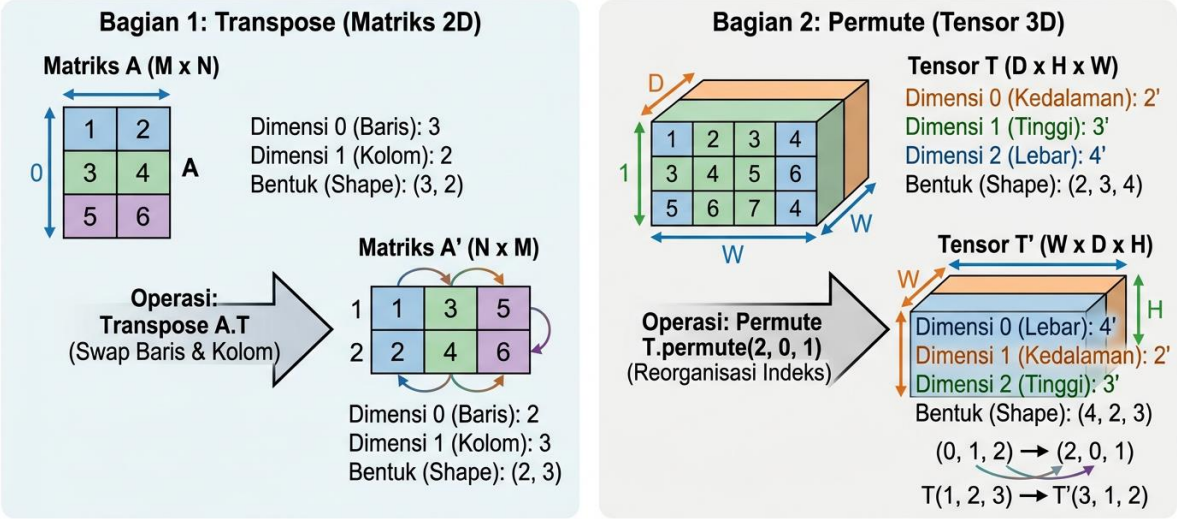
```
# Unsqueeze dan Squeeze
unsqueezed = t.unsqueeze(0)  # Tambah dimensi di posisi 0
print(f"\nUnsqueeze dim=0 : {unsqueezed.shape}")

squeezed = unsqueezed.squeeze(0)  # Hapus dimensi ukuran 1
print(f"Squeeze dim=0   : {squeezed.shape}")
```



Gambar 2.10: Ilustrasi penambahan dan penghapusan dimensi singleton (dimensi berukuran 1) pada tensor menggunakan fungsi `unsqueeze(0)` untuk menambah dimensi baru di indeks ke-0 dan `squeeze(0)` untuk mengembalikannya ke bentuk semula.

CODE

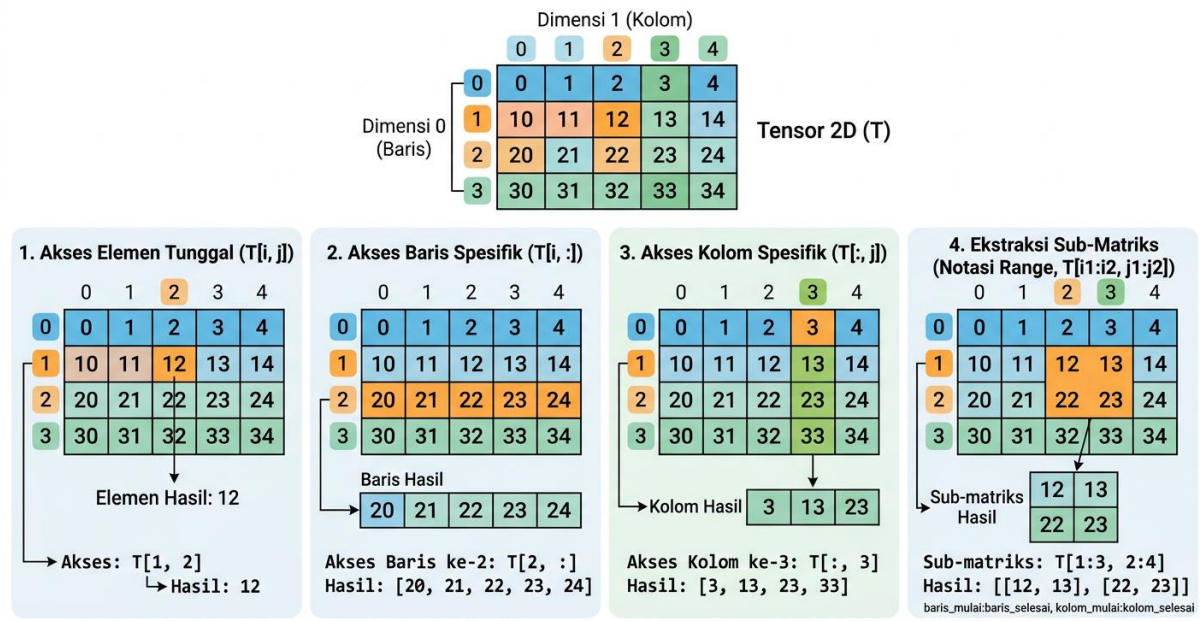
```
# Transpose dan Permute
matrix = torch.rand(2, 3)
print(f"\nOriginal (2x3):\n{matrix}")
print(f"Transpose (3x2):\n{matrix.T}")

tensor_3d = torch.rand(2, 3, 4)
permuted = tensor_3d.permute(2, 0, 1)  # (4, 2, 3)
print(f"\nPermute (2,3,4) -> {permuted.shape}")
```



Gambar 2.11: Ilustrasi reorganisasi dimensi tensor menggunakan transpose (membalikkan baris dan kolom pada matriks 2D) dan permute (pengaturan ulang urutan indeks dimensi pada tensor multidimensi/3D).

CODE

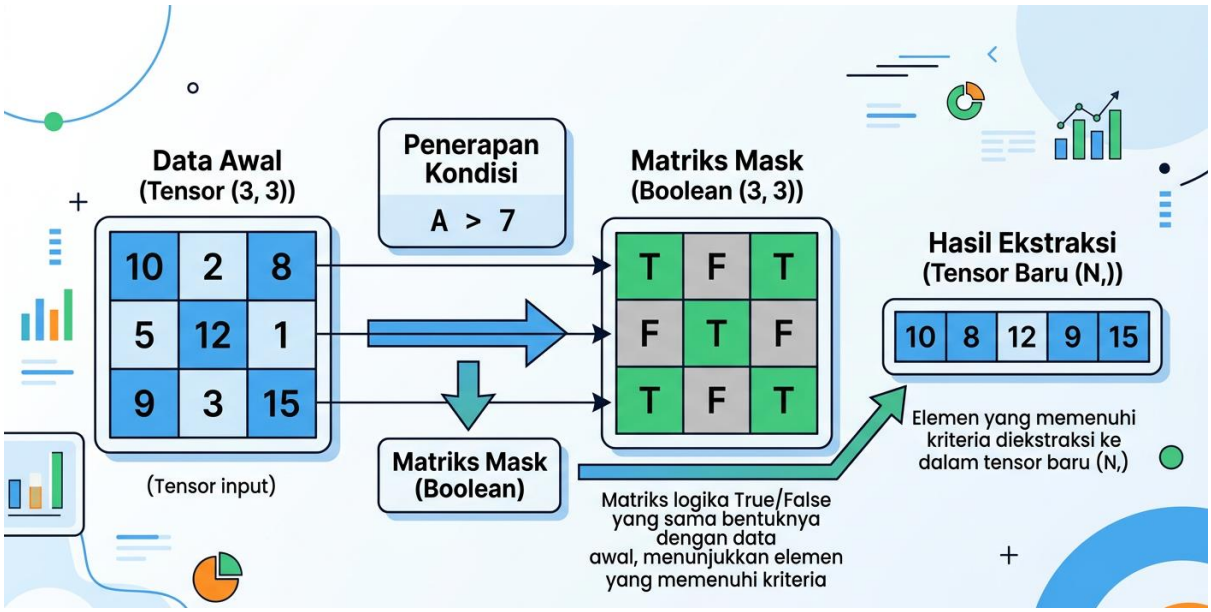
```
# Indexing dan Slicing
m = torch.tensor([[1, 2, 3],
                  [4, 5, 6],
                  [7, 8, 9]])

print(f"\nElement [1,2]     : {m[1, 2]}")
print(f"Row 0             : {m[0]}")
print(f"Column 1          : {m[:, 1]}")
print(f"Sub-matrix        : \n{m[0:2, 1:3]}")
```



Gambar 2.12: Teknik indexing dan slicing pada tensor 2D untuk mengakses elemen tunggal, baris/kolom spesifik, serta ekstraksi sub-matriks menggunakan notasi range.

CODE

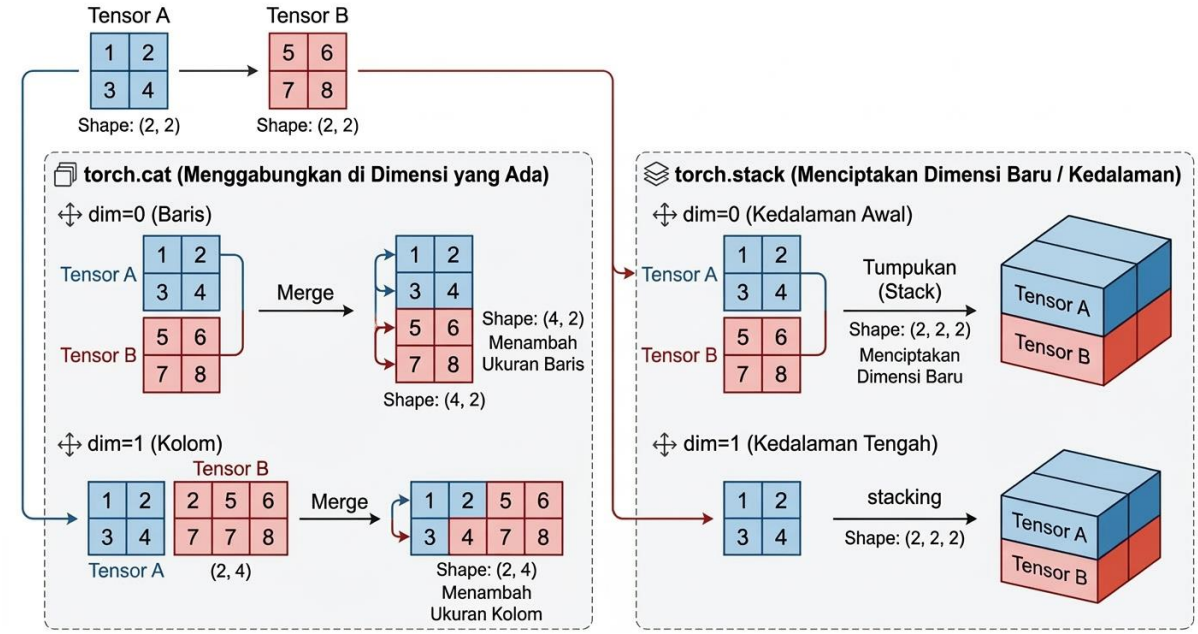
```
# Boolean indexing
mask = m > 5
print(f"\nMask (>5):\n{mask}")
print(f"Filtered  : {m[mask]}")
```



Gambar 2.13: Proses penyaringan data menggunakan Boolean Indexing, yang melibatkan pembuatan matriks mask (logika True/False) berdasarkan kondisi tertentu dan ekstraksi elemen yang memenuhi kriteria tersebut ke dalam tensor baru.

CODE

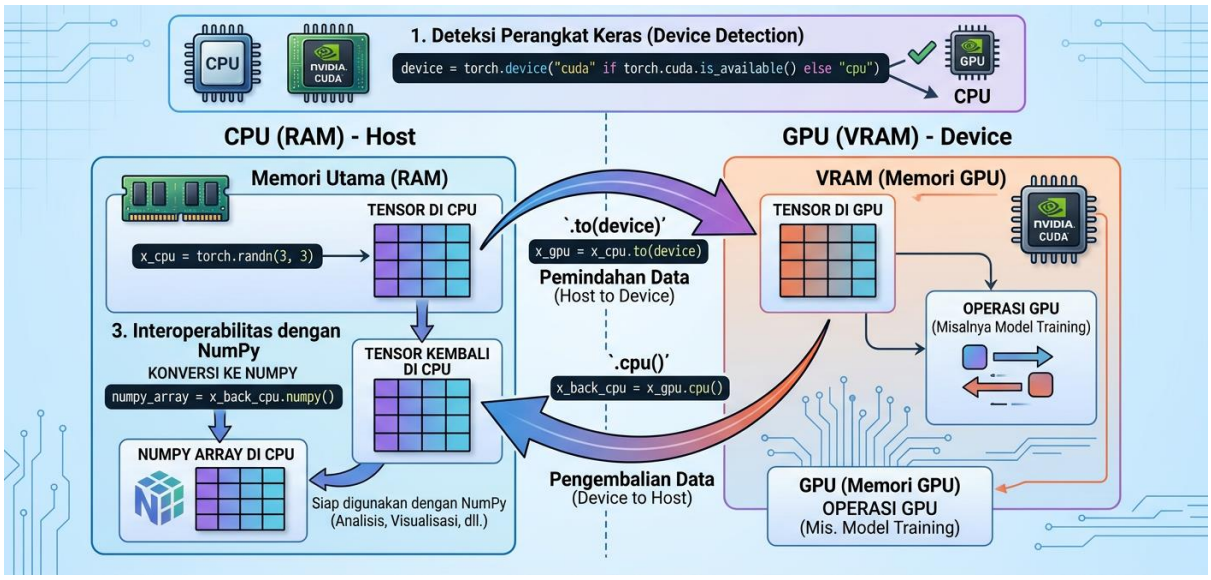
```
# 4. Concatenation dan stacking

t1 = torch.tensor([[1, 2], [3, 4]])
t2 = torch.tensor([[5, 6], [7, 8]])

cat_row = torch.cat([t1, t2], dim=0)
cat_col = torch.cat([t1, t2], dim=1)
stacked = torch.stack([t1, t2], dim=0)

print(f"\nCat dim=0 (row):\n{cat_row}")
print(f"\nCat dim=1 (col):\n{cat_col}")
print(f"\nStack dim=0:\n{stacked} | shape: {stacked.shape}")
```



Gambar 2.14: Visualisasi perbedaan antara operasi torch.cat yang menggabungkan tensor pada dimensi yang sudah ada (baris atau kolom), dan torch.stack yang menggabungkan tensor dengan menciptakan dimensi baru (kedalaman).

CODE

```python
# 5. CPU vs GPU

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

# Membuat tensor langsung di GPU
if torch.cuda.is_available():
    gpu_tensor = torch.rand(3, 3, device=device)
    print(f"\nGPU Tensor:\n{gpu_tensor}")
    print(f"Device: {gpu_tensor.device}")

    # Memindahkan tensor CPU ke GPU
    cpu_tensor = torch.rand(3, 3)
    gpu_tensor2 = cpu_tensor.to(device)
    print(f"\nMoved to: {gpu_tensor2.device}")

    # Operasi di GPU
    result_gpu = gpu_tensor + gpu_tensor2
    print(f"Result device: {result_gpu.device}")

    # Kembali ke CPU (untuk NumPy, plotting, dll)
    result_cpu = result_gpu.cpu()
    result_numpy = result_cpu.numpy()
    print(f"Back to CPU: {result_cpu.device}")
```



Gambar 2.15: Ilustrasi manajemen memori pada PyTorch, mencakup deteksi perangkat keras (device detection), pemindahan tensor dari RAM (CPU) ke VRAM (GPU) menggunakan metode `.to(device)`, serta pengembalian data ke CPU untuk keperluan interoperabilitas dengan NumPy.